Comparison of the data types in Python, covering Numeric, Sequence, Mapping, Set, and Boolean types

Sr no.	Category	Data Type	Description	Mutable/Immutable	Usage	Key Differences
1	Numeric Types	int	Represents integer numbers, both positive and negative, e.g., 5, -3	Immutable	Used for counting, indexing, and mathematical operations	Supports arithmetic operations but no decimal point
2		float	Represents numbers with decimal points, e.g., 5.7, -2.3	Immutable	Used for precise calculations, including scientific and financial computations	Allows decimal values
3	Sequence Types	str	Represents a sequence of characters, e.g., "hello", "Python"	Immutable	Used for handling text and string manipulations	Cannot be modified after creation, supports indexing and slicing
4		list	Represents an ordered collection of items, e.g., [1, 2, 3], ["a", "b", "c"]	Mutable	Used to store multiple items in a single variable with flexibility	Allows modification, adding, or removing elements
5		tuple	Similar to a list, but immutable, e.g., (1, 2, 3), ("a", "b", "c")	Immutable	Used for grouping related data together, providing protection against accidental changes	Once created, cannot be changed
6		range	Represents a sequence of numbers, usually used in loops, e.g., range(5)	Immutable	Commonly used for iteration, creating sequences for loops	Generates numbers in a sequence, evaluated lazily

7	Mapping Type	dict	Stores data as key-value pairs, e.g., {"name": "Alice", "age": 25}	Mutable	Used for storing data that is associated with a unique key, like databases or structured data	Keys must be unique, values can be any type, efficient lookups
8	Set Types	set	Represents an unordered collection of unique items, e.g., {1, 2, 3}, {"apple", "banana"}	Mutable	Used to store unique items, often for membership testing or removing duplicates	Unordered, no duplicates, supports union, intersection, and difference operations
9		frozenset	Immutable version of a set, e.g., frozenset([1, 2, 3])	Immutable	Used in scenarios where a set with unique items is needed but should not be changed	Cannot be modified, still supports union, intersection, and difference
10	Boolean Type	bool	Represents truth values: True or False	Immutable	Used in conditional statements and logical operations	Only two possible values, usually the result of comparisons or logical operations
11	Numeric vs Bool	int vs bool	int can represent many values (whole numbers), while bool represents only True or False	Immutable	Integers used for general numeric operations, bools for logic	Bools are a subclass of int where True=1 and False=0
12	List vs Tuple	list vs tuple	Lists are mutable, whereas tuples are immutable. Lists are slower, and tuples are faster	Mutable/Immutable	Lists for dynamic data storage, tuples for fixed data	Lists can be modified after creation; tuples cannot
13	Set vs List	set vs list	Sets store unique, unordered items; lists store ordered items that can have duplicates	Mutable	Sets for unique collections, lists for ordered and indexable data	Sets don't support indexing; lists do

14	Set vs Dictionary	set vs dict	Sets have unique items without keys, while dictionaries have key-value pairs	Mutable	Sets for membership tests; dictionaries for key-based lookups	Dict keys must be unique; set items must be unique
15	Range vs List	range vs list	Range is lazily evaluated and immutable, lists are fully evaluated and mutable	Immutable/ Mutable	Ranges for sequences in loops; lists for storing data	Range values are not stored until needed; lists store all items
16	List vs Dict	list vs dict	Lists have only values in order, dictionaries store data in key-value pairs	Mutable	Lists for simple ordered data, dictionaries for structured, key-accessible data	Dicts provide key-based access, lists provide index-based access
17	String vs List	str vs list	Strings are a sequence of characters, lists are a collection of any type	Immutable/ Mutable	Strings for text processing, lists for grouping items	Strings can only contain characters, lists can contain any data type
18	Complex vs Float	complex vs float	Floats are real numbers, complex numbers include imaginary components	Immutable	Floats for real number calculations, complex numbers for scientific calculations	Complex numbers include a real and imaginary part, floats only a real part
19	Dict vs Bool	dict vs bool	Dictionaries store multiple key-value pairs, booleans represent True or False	Mutable/Im mutable	Dictionaries for data mapping, booleans for logical statements	Dicts can store complex data structures, bools only represent binary states
20	Int vs Float	int vs float	Integers have no decimal point, floats do	Immutable	Ints for exact counting, floats for precise calculations	Ints are exact, floats can lose precision due to rounding
21	Frozen Set vs Set	frozenset vs set	Frozensets are immutable sets, regular sets are mutable	Immutable/ Mutable	Frozensets for fixed collections, sets for dynamic collections	Frozensets can't be modified after creation

22	Tuple vs String	tuple vs str	Tuples can contain any data type, strings contain only characters	Immutable	Tuples for grouping any data, strings for text	Tuples are heterogeneous; strings are homogeneous (character data)
23	Range vs Tuple	range vs tuple	Range generates sequences, tuples store fixed sequences	Immutable	Ranges for loops, tuples for storing constant sequences	Range generates values on demand, tuple stores them in memory
24	Set vs Bool	set vs bool	Sets can contain multiple values, booleans only hold True or False	Mutable/Immutable	Sets for unique collection storage, bools for logic	Sets are collections; bool is a single binary state
25	Numeric Conversion	int, float, complex	Numeric types can be converted into each other using int(), float(), complex()	Immutable	Enables flexibility in calculations	Conversions between them may cause data loss or approximation
26	Nested Data	list, dict, tuple	Lists, dictionaries, and tuples can store nested structures, e.g., lists within lists	Mutable/Immutable	Common in data organization, e.g., JSON or data tables	Allows creation of complex data structures
27	Membership Tests	list, set, tuple, str	in keyword can be used with lists, sets, tuples, and strings to check if an item exists	Mutable/Immutable	Useful for quick lookups	Sets are fastest for membership tests due to hashing
28	Indexing	list, tuple, str	Lists, tuples, and strings support indexing to access elements	Mutable/Immutable	Useful for accessing specific elements	Sets and dictionaries do not support indexing
29	Ordering	list, tuple, range	Lists, tuples, and ranges are ordered; sets and dictionaries are unordered	Mutable/Immutable	Ordered types preserve element order; unordered types do not	Sets and dicts do not preserve order in all versions of Python

30	Immutability	int, float, str, tuple, frozenset	These types cannot be changed after creation	Immutable	Useful for fixed values or constants	Immutable types provide safer code as data cannot be altered

Credence IT Professional Training Institute